Course repository: https://github.com/amirih/cs-101-2026-spring

**Your Full Name:** .....

# Challenge 1: *Count down*

**Problem:** Starting from a number and going backward by 1 until you reach 0.

**Your goal:** Implement `countdown(n)`, a recursive function that counts down to 1.

**Key idea:**

- **Base case:** when `n == 0`, stop and return
- **Recursive case:** print `n`, then call `countdown(n - 1)`

**Examples** `countdown(2)`: 2

1

**Step by step for `countdown(2)`:**

→ `countdown(2)`: 2, then `countdown(1)`

→ `countdown(1)`: 1, then `countdown(0)`

→ `countdown(0)` returns

**Fill in the blank using these options:**

A) `n+2` B) `n+1` C) `n` D) `n-1` E) `n-2` F) `n*n`

```
def countdown(n):
    if n == 0:
        return

    print(---)
    return countdown(------)
```

# Challenge 1 Solved!

```python
def countdown(n):

    if n == 0:
        return

    print(n)

    return countdown(n - 1)

if __name__ == "__main__":

    countdown(5)
```

```
Output:
5
4
3
2
1
```

| Your result: | | Time Record: | |
|---|---|---|---|
| ☐ 3 | Exactly the same :) | ☐ 1 | Unhappy :( |
| ☐ 3 | Correct, but not identical :) | ☐ 0 | Wrong |
| ☐ 2 | Close | ☐ -1 | Completely wrong |

# Challenge 2: *Factorial*

**Problem:** ***Factorial*** is a mathematical operation that means multiplying all the whole numbers from 1 up to a given number. For example, the factorial of 3 is 3 × 2 × 1 = 6.

**Your goal:** Implement `factorial(n)`, a recursive function that calculates n!.

**Key idea:**

- **Base case:** `factorial(1) = 1`
- **Recursive case:** `factorial(n) = n * factorial(n - 1)`

**Examples:** `factorial(3) = 3 * 2 * 1 = 6`

**Step by step** for factorial(3):

```
factorial(3) = 3 * factorial(2)
             = 3 * (2 * factorial(1))
             = 3 * (2 * 1)
             = 6
```

**<u>Your Solution:</u>**

```
def factorial(n):

    if n == 1:
        return 1

    return --------------------
```

# Challenge 2 Solved!

```python
def factorial(n):
    # Base case
    if n == 1:
        return 1

    # Recursive case

    return n * factorial(n - 1)



if __name__ == "__main__":
    print(f"Factorial of 3: {factorial(3)}")
```

```
Output:
Factorial of 3: 6
```

| Your result: | Time Record: |
|---|---|
| ☐ 3 Exactly the same :) | ☐ 1 Unhappy :( |
| ☐ 3 Correct, but not identical :) | ☐ 0 Wrong |
| ☐ 2 Close | ☐ -1 Completely wrong |

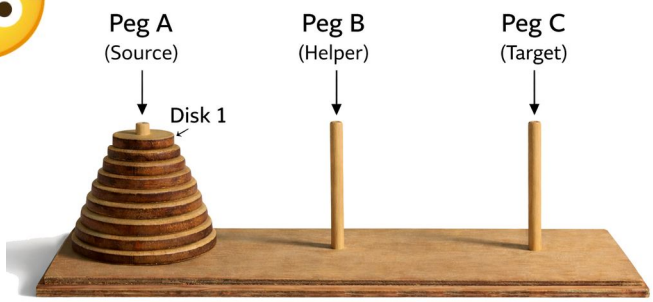
# Organize Your Mind!

**Write at least 50 words about what you learned today. Feel free to include a short summary of the class for your own reference.**

## How are you doing so far?

- ☐ Happy :)
- ☐ Very hard topic but happy
- ☐ OK
- ☐ Unhappy :(
- ☐ Extremely hard topic, sad
- ☐ Nothing to share

# Challenge 3: *Tower of Hanoi*




**Problem:** Move all disks from Peg A to Peg C

**Rules:**

1. Move only one disk at a time
2. You may only move the top disk from a peg
3. Never place a larger disk on top of a smaller disk

**Your goal:** Write a recursive function that moves **n** disks from the source peg to the target peg

## Examples:

| Disks | Minimum moves | Moves |
|---|---|---|
| 1 | 1 | A to C |
| 2 | 3 | A to B, A to C, B to C |
| 3 | 7 | A to C, A to B, C to B, A to C, B to A, B to C, A to C |

## Think recursively (let recursion handle it):

To move **n** disks from **Peg A** to **Peg C**:

1. **Recursively** move the top **n - 1** disks from **Peg A (source)** to **Peg B (helper)**
2. Move the **largest** disk from **Peg A (source)** to **Peg C (target)**
3. **Recursively** move the **n - 1** disks from **Peg B ( helper)** to **Peg C (target)**

In other words, assume the function already works for n - 1 disks. Then ask: how can you use that to solve the problem for n disks? Also, what is the stopping point?

**Tip:** The stopping point is when n = 1, meaning there is only one disk left to move to the target.

**Your Solution:**

```python
def hanoi(n, source, target, helper):
    # Base case
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
```

```python
if __name__ == "__main__":
    num_disks = 3
    hanoi(num_disks, 'A', 'C', 'B')
```

# Challenge 3 Solved!

```python
def hanoi(n, source, target, helper):
    # Base case
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return

    # Recursive case
    hanoi(n - 1, source, helper, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, helper, target, source)


if __name__ == "__main__":
    num_disks = 3
    hanoi(num_disks, 'A', 'C', 'B')
```

```
Output:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

| Your result: | Time Record: |
|---|---|
| ☐ 3 Exactly the same :) | ☐ 1 Unhappy :( |
| ☐ 3 Correct, but not identical :) | ☐ 0 Wrong |
| ☐ 2 Close | ☐ -1 Completely wrong |